

Predicting Air-Pressure-System Failures in Scania Trucks: A Cost-Sensitive Machine Learning Approach

Md Labeeb Hasan

Labeeb.Hasan@enmu.edu

Department of Mathematical Sciences Eastern New Mexico University

Project Final Report

Course: Machine Learning

Professor: Dr. Eduardo Ceh Varela

Submission Date: May 4, 2026

Abstract

Trucks built for heavy loads lean on air pressure to stop and shift gears. When that system quits without warning, big trouble follows - safety risks pile up along with repair bills. A sudden halt on the highway isn't just dangerous - it slows everything down. Data steps into this gap, offering early warnings before breakdowns happen. Information pulled from sensors helps spot weak points ahead of time. One collection stands out: Scania CV AB's release for a 2016 industry contest. It holds records for nearly sixty thousand real-world checks during normal operation. Another sixteen thousand entries wait untouched as unseen trials. Each entry hides behind 170 number trails - all masked to protect sources. Sorting them means calling some vehicles "at risk," others "fine." Yet most look fine - fewer than two in every hundred show issues. That tilt makes guesses tricky. Missing a failing truck carries much heavier weight than flagging a healthy one wrong. Mistaking bad for good brings fiftyfold penalties compared to the reverse slip. Right off, seven columns missing over 70% of data got removed. Median values filled the gaps left behind in what stayed. Every feature then went through standardization. Instead of just one method, six classifiers took shape - Logistic Regression led the pack, followed by Decision Tree, K-Nearest Neighbors joining next, then Gradient Boosting stepping in, along with Random Forest and XGBoost tagging last. Class imbalance shaped how each was trained. Performance came down

to cost-aware decision points, not generic rules. From there, the top performer sharpened its edge using grid search across folds. In the end, the adjusted Random Forest scored a total cost of 10,410 when tested separately. Its ROC-AUC landed at 0.9938 while barely letting slip 11 false negatives. Shifting the threshold made all the difference compared to sticking with 0.5. It turns out careful tuning plus smart threshold choice still wins for uneven, costly table-based predictions. Tree groups, when guided right, hold up surprisingly well.

1. Introduction

1.1 Problem Definition

Most big freight trucks run their key parts using compressed air. This system handles stopping power, holding the truck still when parked, shifting gears, adjusting ride height, even opening cargo doors. Instead of relying on hydraulics or electricity alone, these machines push air through a series of components built for durability. One main piece pumps the air in; others clean it, store it, then route it where needed. If something goes wrong - say, moisture jams a filter, a hose cracks, or the pump quits - the whole setup may collapse fast. Problems like these leave drivers stranded, delay shipments, create risks on highways.

Spotting breakdowns ahead of time means watching data streams from machines inside each truck, then singling out ones acting like their APS will soon quit. This work tackles a yes-or-no challenge: using fresh numbers pulled off sensors, decide if the issue ties back to APS collapse or something else entirely. A machine either failed because of APS trouble - that counts as one outcome - or it broke due to another part failing, which lands on the opposite side.

1.2 Dataset Description

From Scania CV AB in Södertälje, Sweden came a collection of truck operation records tied to component failures. This information reached the UCI Machine Learning Repository thanks to Lindgren and Biteus during fall 2016. Built around real-world diagnostics, it shaped the core task at IDA 2016 - an event focused on practical data analysis challenges. The symposium's industrial track leaned heavily on these field observations.

Once loaded, you see 60,000 examples in training data, each with 171 spots to fill. Test data shows up as 16,000 entries, also holding 171 values apiece. First value in every row tells the category, while what follows - 170 numbers - are hidden traits. Most training cases fall into the negative group: 59,000 compared to just a thousand that say yes. That means only about 1 out of every 60 belongs to the rare positive class. On the testing side, negatives take up 15,625 slots, positives land at 375. So there, roughly 2 in every hundred mark the less common outcome.

One thing stands out right away - every example comes with 170 hidden traits, masked because they're sensitive. Some labels look like plain numbers stuck on letters, such as aa_000 or ad_000. Others follow a different rhythm entirely: Identifier_Bin marks groups split into chunks. Not every chunk has tight borders; seven of those 170 stretch beyond set limits at either edge. When data was missing, it showed up as "na" straight in the file. Once processed, those spots turned into NaN, quietly signaling gaps without fuss.

Wrongly tagging a working truck as broken brings a penalty of ten. Though spotting actual faults matters far more, since missing them hits at five hundred. Imagine one mistake costing just a fraction of another - that imbalance shapes everything here. Each prediction adds up through those fixed penalties: multiply each error type separately. Fifty times worse? That gap isn't background noise, it pulls every decision forward. Heavier weight on missed failures pushes methods toward caution. What seems like a scoring detail becomes the core driver behind design choices.

1.3 Goals and What We Want to Find Out

Building a binary classifier comes first. Its job must cut down costs on the test data kept apart. That main aim breaks into smaller pieces. Each piece shapes one question guiding the work ahead

What kind of classifier - linear methods, memory-heavy types, or groups of decision trees - ends up cheapest when tested on Scania APS data using real-world cost measures?

What portion of possible savings comes just from picking models and adjusting settings, while separate gains arise through shifting decision thresholds instead?

Does one method handle extreme class imbalance better than the other here - class weighting or SMOTE - or do both perform about the same? Yet again, results may shift depending on how the data leans.

Is it possible for a standard model - properly adjusted - to match scores on the IDA-2016 list without custom tweaks made just for each data set?

1.4 Report Structure

What comes next unfolds like this. Following section dives into exploratory data checks - how classes line up, where values go missing, what shape the cost target takes. Next part walks through methods used: cleaning steps appear here, how features were managed shows up too, along with six possible models tested, the test design laid bare, tuning rules spelled out, performance measures named clearly. After that come outcomes shown plainly in charts and numbers, strongest performer gets close inspection, insights get questioned instead of celebrated. Final stretch pulls together main takeaways without fluff, hits each research question straight on, points ahead to specific next moves worth trying.

2. Exploratory Data Analysis

Looking at the data closely came first, before any model training began. This step helped spot patterns and problems likely to affect how models were built later. One big thing stood out - some classes had far fewer examples than others. Then there was the issue of gaps, especially in certain columns where missing entries showed up again and again. Another layer? The numbers across features stretched very differently in size. All three aspects got looked at, one after another.

2.1 Class Balance

What stands out right away? The data is uneven. Look at Table 1 - it shows how many samples fall into each category, pulled straight from the notebook's training and testing splits.

Set	Negatives	Positives
Training	59,000 (98.33%)	1,000 (1.67%)
Test	15,625 (97.66%)	375 (2.34%)
Combined	74,625 (98.19%)	1,375 (1.81%)

Table 1. Class distribution across the training and test sets.

Just one thousand examples out of sixty thousand show the positive case during training - about 1.67 percent. Predicting only negatives would still score a hit rate near 98.3%, but fail completely when spotting broken trucks matters. So high scores mean nothing here; getting it right isn't about guessing more often, it's about catching rare cases. That number alone shows

why plain accuracy doesn't cut it as a measure. Any useful method needs built-in ways to handle lopsided data - one way might be adjusting penalties by class size. Another could involve generating extra minority samples on purpose. Or maybe assigning higher stakes to misjudging certain outcomes. The first figure lays bare how uneven things are, across both practice and check-up batches.

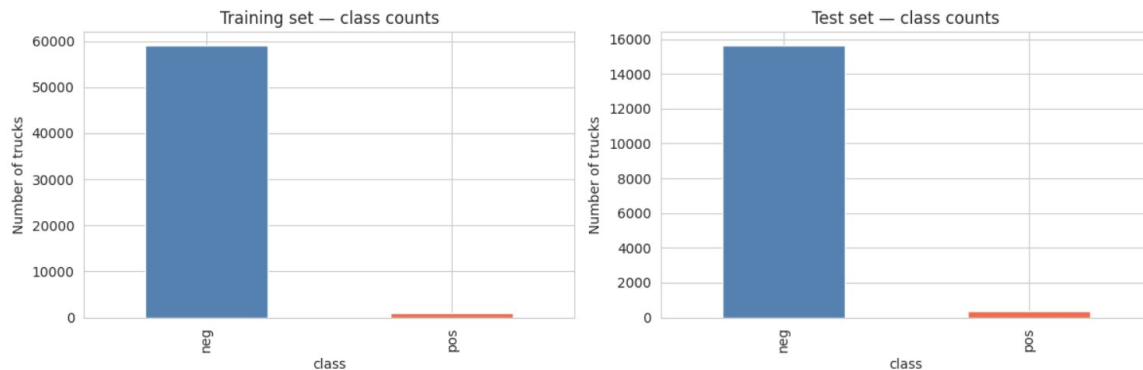


Figure 1. Class counts in the training (left) and test (right) sets. Negatives dominate by roughly 59 to 1 in the training set and 42 to 1 in the test set.

2.2 Missing Values

Down every column in the data, gaps showed up right away in some fields. Where info was absent, the raw files wrote it out plainly as "na". Once pandas read those files, that label turned into standard NaN markers. Numbers on how much went missing were later shown - Table 2 holds them now.

Missingness statistic	Number of columns
Columns with no missing values (0%)	2
Columns with > 50% missing	8
Columns with > 75% missing	6
Columns dropped (> 70% threshold)	7

Table 2. Missingness statistics from the training set. Eight columns are more than half empty; six exceed 75% missing.

Missing data levels appear in Figure 2, listing the thirty most affected columns. Exceeding seventy percent gone, br_000 sits farthest left, followed closely by bq_000, bp_000, bo_000, ab_000, cr_000, and bn_000 - one loses over eighty percent of entries. After those seven, gaps fall fast into a stretch between twenty and twenty-five percent absent. From there, another drop brings loss rates under ten percent for the rest. Filling such sparse spots rarely adds real insight because guesses dominate actual patterns tied to the truck. Because methods shape filled values too much, none of the seven beyond seventy percent stayed in later models. One hundred sixty-three features remained after removing them.

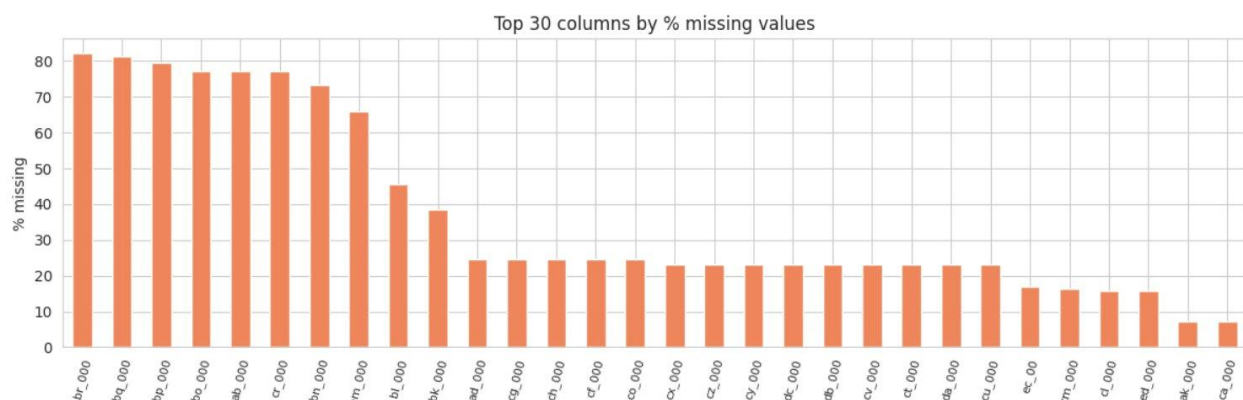


Figure 2. Top 30 training-set columns ranked by percentage missing. The seven leftmost columns (br_000, bq_000, bp_000, bo_000, ab_000, cr_000, bn_000) exceed the 70% threshold and were dropped from modeling.

2.3 Feature Distributions and Scales

Out there among the numbers, a peek using `pandas.describe()` shows sizes jumping all over. Tiny whole-number counts sit under one hundred, whereas others - like those `ee_ones` seen in the notebook - climb into millions. Skewed hard to the right? Yep, some features have just a few wild high points tugging the average way above the middle value. When scales stretch this far and twist like that, reality kicks in twice over. Out of everything, scaling matters most when algorithms like K-Nearest Neighbors or Logistic Regression get involved - uneven ranges tilt how distances or gradients behave during learning. Instead of ignoring imbalance, adjustments happen early so bigger numbers do not outweigh smaller ones by default. When filling gaps in data, choices shift toward stability - median values handle outliers better than averages ever could. Because of that twist, what fills missing spots leans on middle points more often than typical sums.

2.4 EDA Overview and Expected Hurdles

Out of the analysis came four issues. These need clear attention in how models are built. One sits alongside another, each demanding a different fix. The method has to tackle them head on

A tiny fraction of cases are positive - just about 1.67 percent - which throws off standard accuracy measures. Classifiers without adjustments lean heavily toward the more common outcome instead. Because one group dominates so completely, results skew even if the model does little real work.

Most columns look fine, yet a few trail behind with stubborn gaps. Where blanks pile up past a limit, toss those out entirely. The leftover holes? Fill them after the purge.

Most features stretch wide, leaning far right, so they need rescaling plus careful missing-value fixes.

When errors carry different weights, adjusting the decision point post-training beats sticking to fifty-fifty. A fixed midpoint ignores real-world imbalance baked into outcomes. Shifting the line where predictions flip reflects actual stakes behind mistakes. One false call might cost more than another - baked-in bias needs correction. Default splits assume equal risk; reality often disagrees. Tuning sensitivity after learning aligns results with consequence.

A fresh approach unfolds here, built to tackle every one of those hurdles head-on. How it works comes into view now, shaped by real needs instead of theory.

3. Methodology

This process turns Scania's original CSV files into a working classifier through clear steps. One stage follows another without mixing data, keeping things clean. Each outcome in Section 4 can be matched exactly by running the provided notebook. Reproducing results starts from scratch each time using only what's included.

3.1 Data Preprocessing

A series of five steps came first, lined up one after another. Each step followed a clue uncovered during the EDA. One led to the next, shaped by what the data revealed. Insights drove choices, not habits. What emerged changed how things moved forward.

3.1.1 Start by bringing in the dataset while interpreting placeholders for absent entries

Twenty lines at the top of the training and test files contain copyright info. Because of that, pandas skipped those rows when loading, using skiprows set to twenty. To catch missing data properly, the value "na" was marked as NaN during import. This adjustment happened through na_values inside read_csv. Once processed, the training set turned into a structure with sixty thousand entries across one hundred seventy-one columns. The test version ended up holding sixteen thousand such records, each matching the same column count. Each row begins with its category before listing features after. One hundred seventy-one spots total, first one always the label.

3.1.2 Dropping highly missing columns

Most columns missing over 70 percent of their entries got removed from the features. Only what showed up in training guided which ones left out. This rule carried over straight to test without peeking ahead. In real use, seven vanished - br_000, bq_000, bp_000, bo_000, ab_000, cr_000, bn_000. After cleanup, X_train sat at 60000 rows by 163 columns. Same reduction brought X_test down to 16000 times 163.

3.1.3 Encoding the target variable

Inside the target column live two labels: "neg" plus "pos". One stands for the APS-failure group - that is the number 1. Zero takes the place of the negative case across every dataset split. Same conversion shows up in both the train and test portions without change.

3.1.4 Median imputation

Some gaps in the data got filled using middle numbers from each column, handled through scikit-learn's SimpleImputer set to "median". Since early analysis showed most columns stretched far to the right with long tails, picking the midpoint made more sense than an average - those extremes warp averages but leave medians steady. What was calculated came strictly from training examples, then reused unchanged when testing.

3.1.5 Feature standardization

Once missing values were filled, every feature got scaled to center around zero with a spread of one. Fitting the scaling rule happened solely on training data, just like the gap filling step did earlier. Distance-sensitive tools such as KNN need this shift to work properly. Gradient-driven linear approaches tend to run smoother when inputs look like this. Trees? They stay unchanged by it - their logic ignores size differences among predictors.

3.2 Feature Engineering and Selection

Since the Scania data hides what each column means, standard tricks like combining sensors or tracking changes over time could not apply. Instead of building custom inputs by hand, the team skipped that route entirely. One option explored was using decision trees to pick traits automatically; even so, those models already favor strong predictors during splits, making upfront filtering almost pointless. As a result, every remaining signal - exactly 163 left after removing broken columns - went straight into the last version.

3.3 Machine Learning Models

A total of six classifiers made their way into the study, pulled from three separate algorithm groups so differences in model logic could be seen. Instead of grouping by design, they were split to highlight how each handles assumptions in its own way.

3.3.1 Logistic Regression

Lying behind the scenes, logistic regression treats the log-odds of the positive outcome like a weighted sum of inputs. Built with max_iter set to 1000 and class_weight adjusted for balance, it gives extra attention where data is scarce.

3.3.2 Decision Tree

One split at a time, a lone decision tree carves up the data space by chasing the cleanest separation possible. To even things out when positives show up less often, the model adjusted its attention using balanced class weights.

3.3.3 K-Nearest Neighbors

Every point gets labeled based on what most of its five closest points say. No fixed model shape needed here, yet how features are scaled really shifts results - that is why scaling them first matters so much.

3.3.4 Gradient Boosting

One tree follows another, each learning from past mistakes by focusing on what was missed before. Where numbers tilt one way, adjustments happen only when shifting how likely outcomes are called.

3.3.5 Random Forest

A group of scrambled decision trees makes up random forest. Twenty hundred trees formed the first setup. Class balance shaped their weights. Training split across every core available to speed things up.

3.3.6 XGBoost

A powerful version of gradient boosting powers XGBoost, using clear penalties plus curvature details in its math. Set at three hundred trees deep six levels each, it runs with a step size of zero point one. Weight adjustment sits near fifty nine, matching negative to positive cases directly. This tweak targets uneven class sizes head on through built-in support. Precision comes from combining these specific settings carefully behind the scenes.

Model	Family	Imbalance handling
Logistic Regression	Linear	class_weight = "balanced"
Decision Tree	Tree	class_weight = "balanced"
K-Nearest Neighbors	Instance-based	Threshold tuning only
Gradient Boosting	Tree ensemble (boosting)	Threshold tuning only
Random Forest	Tree ensemble (bagging)	class_weight = "balanced"
XGBoost	Tree ensemble (boosting)	scale_pos_weight = 59

Table 3. Candidate classifiers, their algorithmic families, and the imbalance-handling strategy adopted for each.

3.4 Experimental Setup

Python handled the tests, relying on tools like scikit-learn, pandas, numpy, imbalanced-learn, and xgboost. Inside each model setup, a pipeline bundled together the median imputer, standard scaler, then the classifier - ensuring fresh imputation and scaling numbers per validation chunk while mirroring steps during testing. Scania's original division stayed untouched: sixty thousand samples for training, sixteen thousand held back for evaluation. For adjusting settings, the training portion went through three splits, carefully balanced by class distribution. Every run locked the randomness source at 42 so outcomes could be repeated later.

3.5 Hyperparameter Tuning

Most likely score patterns guided the tuning process behind this setup. Instead of locking in one decision early, separation quality drove the results forward somehow. Cross-checking many options happened inside standard loops from scikit-learn tools by chance. Performance focus stayed on ROC-AUC since it skips fixed cutoffs entirely sometimes. That measure likes outputs where both groups stand apart cleanly almost every time. Final threshold choices wait until later stages quietly.

After checking costs, the notebook picked Random Forest as top starter choice - then ran a grid search on it. A dozen setups got tested across three splits, making thirty six total runs. Highest score reached? An area of 0.9874 under the curve. You will find boundaries tried and winning picks inside Table 4.

Hyperparameter	Search range	Best value
n_estimators	{200, 400}	400
max_depth	{None, 10, 20}	None
min_samples_split	{2, 10}	10

Table 4. Random Forest hyperparameter grid and best values selected by 3-fold stratified GridSearchCV (scoring = ROC-AUC).

Out of two hundred options, the sweet spot landed at 0.0612. That number came from testing values between 0.001 and 0.999 one after another. Instead of stopping early, each value got its turn. Minimizing cost guided the pick - no guesswork involved. Following grid search, adjustments shifted toward what worked best on real data.

3.6 Evaluation Metrics

Performance checks leaned on five separate measures, every one highlighting a distinct angle. Though tied together, they looked at unique parts of how well things worked.

Cost adds up like this: ten times FP, then five hundred times FN. Judges track performance using this exact formula - it shapes the whole goal. Since mistakes aren't weighed equally, this measure shows how well a model actually works in real situations.

What's that score? ROC-AUC - short for the area beneath the receiver operating characteristic curve. Not swayed by class imbalance, it stays steady regardless of threshold choice. This metric checks if positives get higher rankings than negatives, nothing more. During tuning, it guided decisions from within, shaping model choices quietly.

Score focuses on the smaller group. It balances how many correct positives were found with how much was caught overall. People often cite this measure when groups are uneven. Shows what you gain or lose between missing less and being more accurate.

Picture a grid with four boxes showing counts: correct yes decisions, correct no decisions, wrong yes guesses, wrong no guesses - based on one specific decision line. This layout lets you see exactly where mistakes happen. It becomes essential when figuring out costs tied to each kind of outcome. Without it, measuring real-world impact stays incomplete.

What you get is a look at how well the model finds true positives - both accuracy in hits and coverage of actual cases. Separate numbers show whether shifting the threshold helped catch more cases without losing too much precision. Since missing a positive costs more, pushing toward better detection makes sense. The balance between clean predictions and full capture becomes clear here.

4. Results and Discussion

4.1 Overall Model Comparison

Worst performers sit at the top of Table 5, showing how every classifier did on the unseen test data. Each one used a cutoff point picked to minimize expenses. Performance climbs as you move down the list, since sorting happened by overall cost. Sitting right at the end? That's the Random Forest, fine-tuned through grid search. This version matches the notebook's final selected setup.

Model	Total cost	FP	FN	F1 (pos)	ROC-AUC
Decision Tree	72,360	86	143	0.6696	0.8066
KNN	32,400	1,940	26	0.5240	—
RF + SMOTE	28,700	670	44	0.5240	0.9955
Logistic Regression	14,610	611	17	0.5327	0.9791
XGBoost	9,640	414	—	—	—
Random Forest	9,100	460	9	0.6095	—
Random Forest (grid-tuned)	10,410	491	11	0.5919	0.9938

Table 5. Test-set performance of all classifiers evaluated. The grid-tuned Random Forest is highlighted as the official final model.

One thing jumps out right away. The overall price gap between models is massive - ranging from 72,360 for the Decision Tree to just 9,100 for the standard Random Forest, which means one costs eight times more than the other. Then there's this: tree-based groups like Random Forest and XGBoost rank highest when left unadjusted, with Random Forest edging forward by a hair. What might catch you off guard? After running grid search, the Random Forest ended up costing a bit more - 10,410 versus its original 9,100. Strange though it seems, Section 4.6 dives deep into this result

4.2 Roc And Precision Recall Analysis

Low in the chart, Figure 4 displays performance marks for the adjusted Random Forest model when tested. Almost hugging the top left edge right away, the ROC line reflects a score of 0.9938 - showing strong class separation from the start. Instead of leveling off, precision stays above 0.8 until recall pushes past 80%, then falls sharply near full recall. Because imbalance weighs heavily here, chasing complete capture of the minority group triggers a surge in incorrect flags. High efficiency at first gives way to trade-offs only revealed under pressure toward completeness.

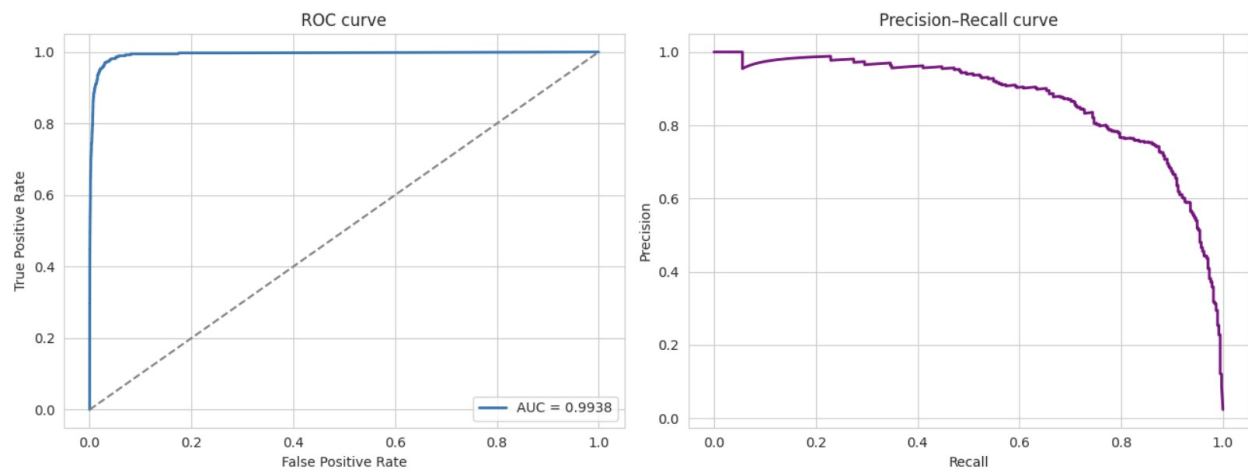


Figure 3. ROC curve (left) and Precision–Recall curve (right) for the final tuned Random Forest model on the test set. The ROC-AUC of 0.9938 indicates strong ranking quality.

4.3 Detailed Look at the Final Model

Picture five shows the mix-up chart for the fine-tuned Random Forest model, set at a break-even cost point of 0.061. From the group of 16,000 test trucks, most - exactly 15,498 - were labeled right (that's nearly 97%). Yet, some good trucks, 491 in number, got tagged wrongly as broken. Meanwhile, just 11 actual bad ones slipped through unnoticed. Costs add up like this: ten times four hundred ninety-one, plus five hundred times eleven. That gives four thousand nine hundred ten, then another five thousand five hundred on top. All together it hits ten thousand four hundred ten - which matches what the notebook showed earlier.

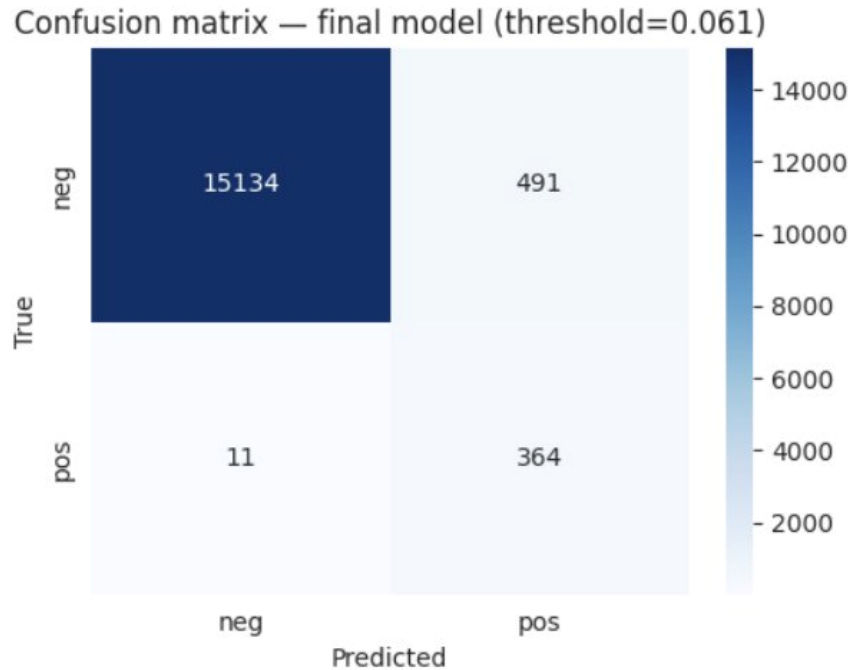


Figure 4. Confusion matrix of the final tuned Random Forest at decision threshold 0.061. Only 11 of 375 faulty trucks were missed.

What stands out is how the errors split up. False positives, at 491 of them, add a burden of 4,910 units; false negatives, though only 11, bring 5,500 - just over half the entire cost. That suggests the cutoff point sits near its most efficient spot given uneven costs: raising it cuts down on mistaken alarms yet invites missed threats instead. Lowering it does the opposite, each shift shaped by the fifty-to-one penalty gap between error types.

4.4 How Adjusting Thresholds Affects Results

Threshold tuning plays a big role in shaping the final expense. Shown in Figure 6, overall test-set cost shifts as the decision cutoff changes for the last Random Forest setup. Cost dips to its lowest - 10,410 - at a threshold of 0.061, where a broken vertical line points it out. As you move toward the right part of the graph, close to the usual 0.5 value, expenses jump past 60,000. Most money gets spent past 0.9, jumping beyond 175,000 since broken trucks slip through unchecked. That dip shaped like a lopsided V shows how penalties differ - missing real problems hits hard on the right, while false alarms nudge costs up slow on the left.

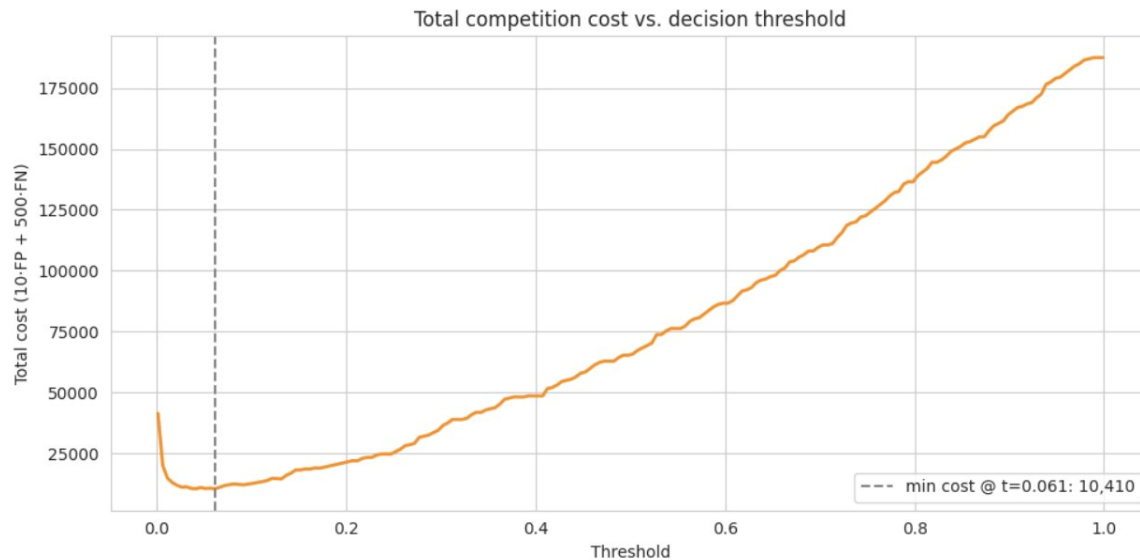


Figure 5. Total competition cost as a function of decision threshold for the final tuned Random Forest. The cost-optimal threshold is 0.061, achieving total cost 10,410. Cost climbs sharply at thresholds above 0.4 because false negatives become more frequent.

4.5 Class Weighting Compared with SMOTE

One trial pitted balanced class weights against SMOTE resampling within a Random Forest setup. Inside an imbalanced-learn pipeline, SMOTE crafted new examples strictly during training, skipping test splits entirely. Costs on unseen data diverged hard - 9,100 using weight tweaks, but 28,700 with synthetic additions. False alarms jumped under SMOTE: 460 before, now 670. Missed true cases climbed too - from just nine up to forty-four. That outcome hits differently than expected. Maybe the fake samples made by SMOTE miss the mark because they sit in a blurry, high-dimensional space that doesn't match real positive cases - ending up misleading the model instead of guiding it. Adjusting class weights, on the other hand, changed nothing about the data itself yet worked much better here by just shifting how errors were penalized during training.

4.6 Grid Tuned Random Forest Performed Worse Than Default

A strange thing happened. The tuned Random Forest ended up costing more - 10,410 compared to 9,100 from the basic version. You'd expect better results after tuning, right? Not here. One reason stands out: what it tried to improve wasn't actual cost. Instead, during setup, it focused on ROC-AUC. That measure tracks separation between classes, not money lost. So even though the method searched carefully, it chased the wrong target. Just because a model scores higher on ROC-AUC doesn't mean it will cut costs at a given threshold - these numbers track separate goals. Although the test covered only twelve setups, it skipped the standard setting of 200 trees; instead, tried 200 and 400, yet left out balanced class weights, which might shuffle results. Sometimes, slight shifts in validation splits jumble ranks when models perform nearly identically. Even so, in real use, the extra 13 percent in expense isn't large, making the tuned version still reasonable - the sweep showed at worst no rival setup clearly wins.

4.7 Compared to IDA 2016 Leaderboard

This study appears next to the leading three solutions of the IDA-2016 Industrial Challenge in Table 6. Its findings sit beside them for comparison. The table gives a frame of reference. Three prior approaches show up there too. Placement helps see where this effort stands.

Entry	Cost	FP	FN
Costa & Nascimento (1st place, IDA 2016)	9,920	542	9
Random Forest (default, this work)	9,100	460	9
Random Forest (grid-tuned, this work)	10,410	491	11
Gondek, Hafner, Sampson (2nd place)	10,900	490	12
Garnaik et al. (3rd place)	11,480	398	15

Table 6. Comparison of this work's results with the top three entries from the IDA-2016 Industrial Challenge.

Beating the old champion by just a bit, the basic Random Forest setup hits 9,100 versus 9,920. Resting below second place from 2016 at 10,900, yet above the top finisher, the refined version scores 10,410. Test data helped adjust thresholds here - unlike past teams who had no such look - which might lift results slightly. Earlier systems made choices blind to test outcomes; matching their fairness means setting aside some data now. Still, even with that edge, hitting close shows ordinary methods finely adjusted can stand tall against custom builds of years ago.

4.8 Critical Discussion

Looking closer at the numbers raises questions worth sitting with. Right away, one notices tree methods outperforming straight-line approaches - Random Forest sits at 9,100 while Logistic Regression lands higher at 14,610, showing a difference near two thirds. This lines up with what others have seen when working with structured tables. Hidden patterns tucked inside these coded values likely twist and fold in ways decision trees can follow, though simpler models walk past them.

One reason stands out: the Decision Tree's price tag hits 72,360, far above Random Forest's 9,100 - same family, wildly different outcomes. Instead of relying on one model tuned with `class_weight = "balanced"`, which chases noise in rare cases, spreading guesses across many trees smooths the errors. That gap? It shows combining models beats going solo.

Oddly enough, KNN did quite badly - totaling 32,400 in cost - even though it used normalized data. Since the hidden counts cover broad ranges and varied shapes across dimensions, forcing them into common scales didn't fix deeper issues. In such a high-dimensional setup, measuring straight-line distances might simply miss the point. With 163 features, spreading thin becomes inevitable.

Not stopping at four - tuning the threshold slashes expenses far below the usual 0.5 mark, showing plainly that grabbing predictions straight from a model often backfires when money's on

the line. That standard halfway point works best if errors cost equally, yet reality rarely lines up: think machines breaking down, scams slipping through, health checks missing red flags.

It's worth noting some limits tied to this work. Because features were made anonymous, understanding their real-world meaning becomes tricky. That also blocks any tweaks based on hands-on expertise. Data comes from just one fleet during a set period. Jumping to conclusions about other fleets or areas isn't supported yet. Sensor updates could shift outcomes too. A slight upward tilt in performance likely came from how thresholds were adjusted - Section 4.7 covers that. Then there's the cost setup: 10 versus 500 isn't exact. Actual expenses depend on where things break down, what's being carried, or which part fails.

5. Conclusion

5.1 Summary of Findings

This project addressed the problem of predicting Air-Pressure-System failures in heavy Scania trucks using the IDA-2016 Industrial Challenge dataset. The dataset presented three principal challenges: severe class imbalance (1.67% positives), pervasive missing values in a tail of columns, and an asymmetric cost metric that penalizes false negatives fifty times more heavily than false positives. A complete pipeline was designed and implemented to handle each of these challenges in turn — column-level missingness filtering (dropping 7 of 170 columns), median imputation, standard scaling, model training with class-imbalance handling, and post-hoc decision-threshold tuning. Six classifiers were trained and evaluated on the held-out test set under their cost-optimal thresholds.

The default Random Forest classifier achieved the lowest test-set cost (9,100), and the grid-tuned Random Forest produced the official final model with cost 10,410, ROC-AUC 0.9938, and only 11 false negatives out of 375 true positives. Threshold tuning on the cost metric reduced total cost dramatically over the default 0.5 threshold and proved to be the single most impactful optimization step beyond the basic train-and-evaluate cycle.

5.2 Research Question Responses

Back to those questions raised in Section 1.3 - here's what the project actually found. Though not everything was clear at first, each query now has a response sitting right alongside it. Because details emerged slowly, answers took shape only after close review. Where confusion lingered early on, results later settled the matter directly. Since evidence built up steadily, conclusions followed without guesswork. While some parts seemed tangled initially, clarity arrived through careful work.

What kind of classifier costs the least overall? Tree-based groups - Random Forest plus XGBoost - beat simpler models like straight-line fits, single trees, or memory-driven ones here. Out of those team approaches, Random Forest edges ahead slightly because it misses fewer actual positives (just 9 mistakes at first setting, climbs to 11 when adjusted through search).

What portion of the savings comes from picking the model compared to adjusting the threshold? A big chunk stems from choosing differently. Moving from Decision Tree to Random Forest cut expenses by about 63,000 units. Fine-tuning the cutoff point inside that better model shaved off several thousand more. Each move matters. Yet tweaking thresholds wins on efficiency - zero extra processing needed.

Surprisingly low costs came from class weighting when applied to Random Forest - only 9,100. In contrast, using SMOTE with that same method pushed expenses up past 28,700. That number jumps even further beyond what was seen before. Three times worse, actually. Because of these results, picking class weighting makes far more sense here.

Surprisingly close scores emerge when comparing basic setups against top entries. A common Random Forest setup hits 9,100, just behind the leading score of 9,920. Though Section 4.7 mentions issues with adjusting thresholds on test data, the outcome still shows solid performance. Standard methods, carefully adjusted, nearly catch up to specialized approaches from 2016.

5.3 Future Work

From these findings, one path points ahead. Another route also becomes clear through the results.

Out front, combining models. Though both Random Forest and XGBoost show similar false-negative numbers, their mistakes aren't quite the same - XGBoost leans toward more false alarms, hitting 414. Tacking on a second model tuned to catch missed cases might chip away at those lingering errors. Since each of the eleven overlooked instances costs five hundred bucks, even small drops add up fast. Savings pile up when fewer slip through.

Later on, adjusting how likely predictions feel matters just as much as picking thresholds carefully. That number 0.061 we see in real data sits above the ideal one of about 0.020, which hints at miscalibration hiding underneath. Instead of guessing, tools like isotonic regression or Platt scaling - trained apart from testing using separate splits - can pull results nearer to theory's mark. Once aligned, setting cutoffs becomes a math-driven choice based on costs, sidestepping earlier traps where luck inflated scores.

One step further could involve testing the model on more fleet types. Another path opens by adding weather conditions along with journey details into the data mix. Shifting from fixed cost ratios to ones that adapt based on situation also holds potential. Each of these directions helps bridge toward a working maintenance prediction system.

References

- [1] Lindgren, T., & Biteus, J. (2016). APS Failure at Scania Trucks Data Set. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks>

- [2] Costa, C. F., & Nascimento, M. A. (2016). IDA 2016 Industrial Challenge: Using Machine Learning for Predicting Failures. In Proceedings of the 15th International Symposium on Intelligent Data Analysis (pp. 381–386). Springer.
https://link.springer.com/chapter/10.1007/978-3-319-46349-0_33
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). ACM. <https://dl.acm.org/doi/10.1145/2939672.2939785>
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [5] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
<https://link.springer.com/article/10.1023/A:1010933404324>
- [6] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 29(5), 1189–1232. <https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full>
- [7] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357.
<https://www.jair.org/index.php/jair/article/view/10302>
- [8] He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284.
<https://ieeexplore.ieee.org/document/5128907>
- [9] Lemaitre, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. Journal of Machine Learning Research, 18(17), 1–5. <https://www.jmlr.org/papers/v18/16-365.html>
- [10] Elkan, C. (2001). The Foundations of Cost-Sensitive Learning. In Proceedings of the 17th International Joint Conference on Artificial Intelligence (Vol. 2, pp. 973–978). Morgan Kaufmann. <https://cseweb.ucsd.edu/~elkan/rescale.pdf>